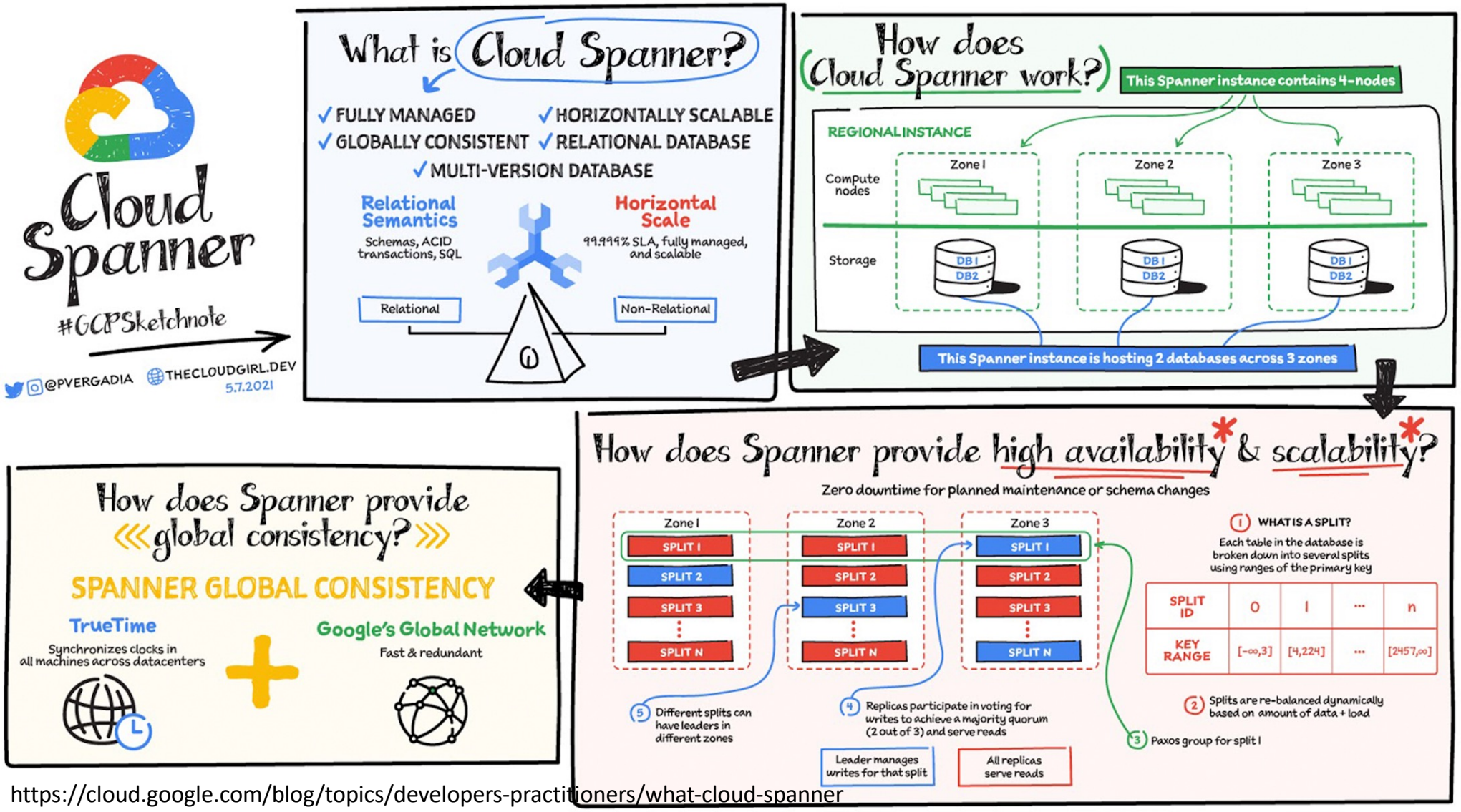
CompSci 390-03-26S

Distributed and Networked Systems

Distributed Transactions



https://cloud.google.com/blog/topics/developers-practitioners/what-cloud-spanner

ACID transactions

A **transaction** executes a sequence of reads/writes on different data items.

```
BEGIN T1
  read X
  read Y
  …
  write X
COMMIT
```

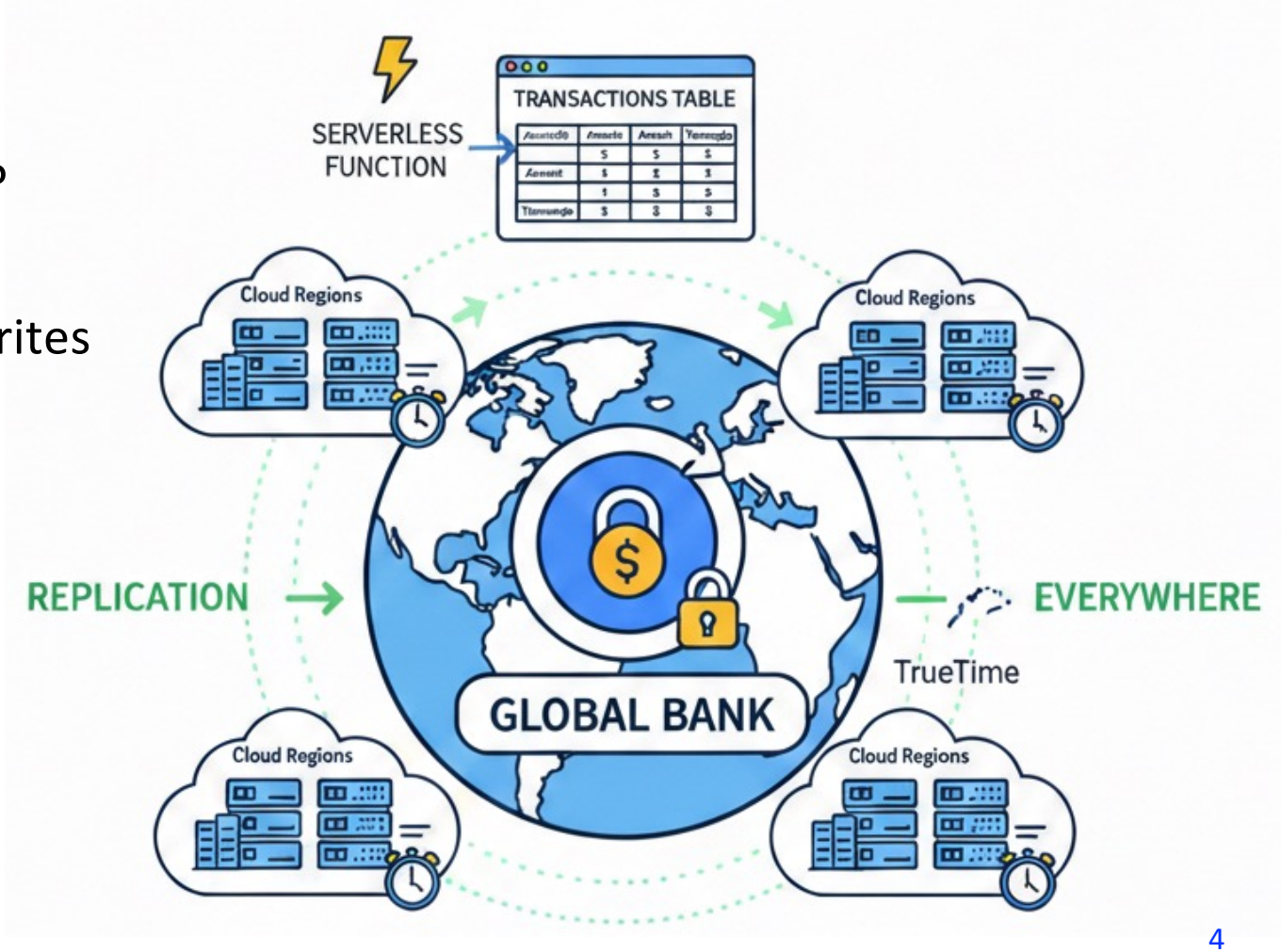
```
BEGIN T2
  read X
  write Y
  …
  write X
COMMIT
```

Atomic
Consistent
Independent
Durable

Transactions **appear** to commit (complete) **atomically** in a serial order.

Global bank

- Numbered accounts
- Grouped in branches?
- Sharded
- Complex reads and writes
  - Balance queries
  - Transfers
  - Cross-shard



Gemini drew it

Isolation and Serializability

- "I" means that actions are *serializable*.
  - A schedule for a group of transactions is *serializable* iff its effect is the same as if they had executed in *some* serial order.
  - Obvious approach: execute them in a serial order (slow).
- Transactions may be interleaved for concurrency, but this requirement constrains the allowable schedules:
  - A transaction must not affect another that commits before it.
  - T_1 and T_2 may be arbitrarily interleaved only if there are no *conflicts* among their operations.
  - Concurrent operations **conflict** if they access the same data item and at least one of them is a write.
  - Conflicts matter: "I" property says that intermediate effects of *T* must be invisible to other transactions unless/until *T* commits.

Conflicts

Consider two transactions *S* and *T*:

T: transfer $100 from *A* to *C:*

S: compute total balance for *A* and *C*:

S and *T* both access *A,* and *S* writes to *A*

→ *S* and *T* **conflict** on *A*.

S and *T* both access *C* and *S* writes *C*

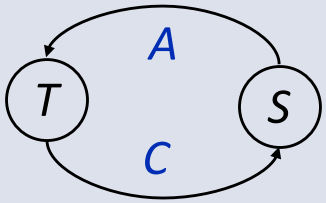
→ *S* and *T* **conflict** on *C*.

Conflict: shared location, at least one writes.

It's a **race**. Who wins? Transactions are serializable if **winner takes all**.

R(A) *W(A)* *R(C)* *W(C)*
R(A) *R(C)*

T: *R(A)* *W(A)* *R(C)* *W(C)*
S: *R(A)* *R(C)*



(*S* total balance gains $100.)

S wins race to A; T wins race to C. **X**

Conflicts

Consider two transactions *S* and *T*:

T: transfer $100 from *A* to *C:*

S: compute total balance for *A* and *C*:

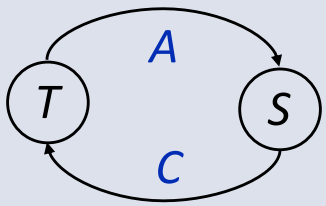
R(A) *W(A)* *R(C)* *W(C)*
R(A) *R(C)*

The directed graph shows a vertex for each transaction, and an edge for each object.

Direction reflects "wins": precedence order of conflicting accesses W(*x*) and R(*x*).

No cycles? → serializable

T: *R(A)* *W(A)* *R(C)* *W(C)*
S: *R(A)* *R(C)*



(*S* total balance loses $100.)

T wins race to A; S wins race to C. **X**

Legal schedules

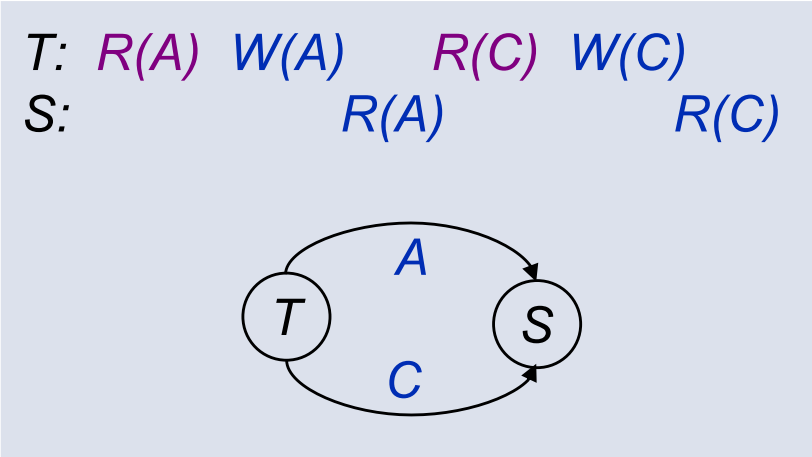
"Winner take all" rule means either:

- S<T for all conflicting accesses
- T<S for all conflicting accesses

→ no cycles

- Such a schedule is **conflict-serializable**.
- It is "equivalent" to some serializable schedule.

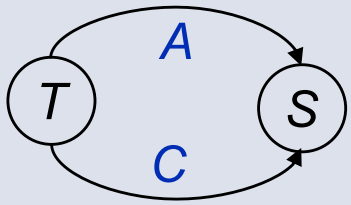
→ It passes the ACID test.



Managing conflicts

- Transaction serializability is a problem of **concurrency control**.
  - "Transactions are threads."
  - Multiple options, but we can use our old tools: locks.
  - But when to release?
- **Two-phase locking (2PL)**: Each T acquires all its locks before releasing any.
- → There is a point in time | when *S* and *T* each hold all their locks.
- These points define the < ordering.

T: *R(A)* *W(A)* | *R(C)* *W(C)*
S: | *R(A)* *R(C)*



Transactional Concurrency Control

Three ways to ensure serial-equivalent order on conflicts:

- Option 1, execute transactions serially.
- Option 2, **pessimistic concurrency control**: block *T* until transactions with conflicting operations are done.
  - Use locks for mutual exclusion
  - **Two-phase locking (2PL)**
- Option 3, **optimistic concurrency control (OCC)**: proceed in hopes it will all be conflict-free. If there's a collision, fix it up.
  - Validate transactions before allowing them to commit.
  - If a transaction T could have observed any conflict with transaction S that violates serializability, abort/restart T or S or both.

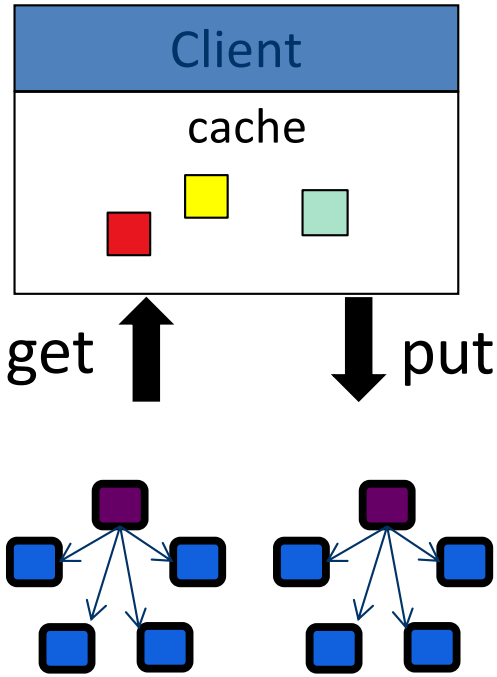
Client/server transactions

Design pattern* for cloud transactions (e.g., Spanner).

- Each transaction T executes on a single **client**.
- T operates on keys/objects in client **cache**.
- Storage tier: shard **servers** are **replica groups**.

Client for T:

- App code for each T runs on **one client**—not servers.
- Client maintains object cache in its memory.
- Read/get keys/objects into cache from the shards.
- Buffer T's updates in cache; write back on **commit**.



Client-server transactions introduced in Thor (Liskov and Adya etc. @MIT 1995)

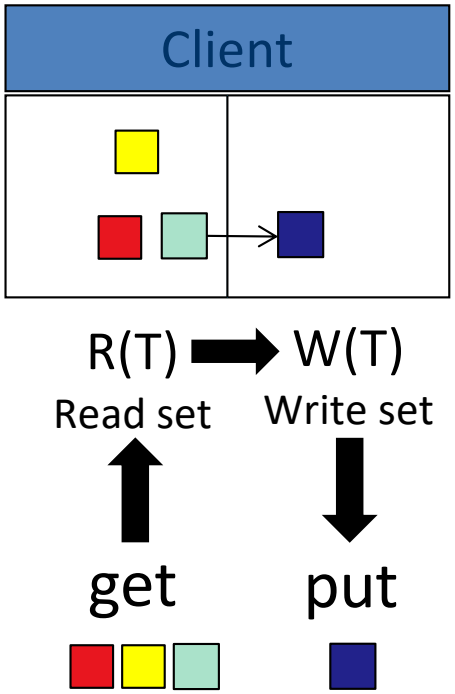
Client/server transactions: closer look

Client tracks its local cache state for T:

- **Read set R(T)**: objectIDs (keys) and versions read by T
- **Write set W(T)**: "dirty" objects modified by T
- W(T) ⊂ R(T): no "blind writes"

On commit(T) client coordinates **two-phase commit** (2PC):

- **Phase 1**: request **prepare(T)** to all involved shards
  - Pass R(T) and W(T) subsets for each shard
  - **Push:** put new values for W(T) to their shards
- **Phase 2**: if all shards agree: **decide**→T will commit
  - Else if any shard rejects, abort T
  - Notify shards of decision asynchronously



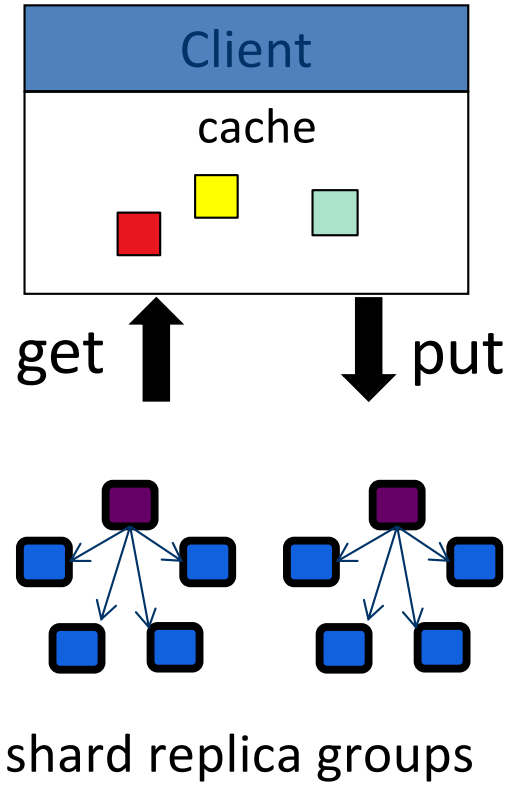
"Spiffle": simplified client/server transactions

A "simple" transactional KV service

- Client-server: each transaction runs on one client.
- Suppose transactions are serial at each client.
  - *Waves hands*: we have locks, we know what to do.

Interesting parts to discuss: commit!

- **2PC**. Client coordinates 2PC to shard servers
- **OCC**. Optimistic with version timestamps and commit-time validation per-shard
- **2PL**. Commit-time 2PL at shards for concurrent prepares distributed across shards



Roots of Spiffle

- **Transactions** with multiple servers and two-phase commit (2PC) have deep roots in databases from a distant past (1970s).
- **Optimistic concurrency control**: validate T on commit, abort/restart if it happens to conflict with another transaction S. (OCC, HT Kung, 1979)
- **Thor**. Client/server transactions with OCC, sharding, loosely synchronized clocks, and and external consistency. (Liskov/MIT 1995)
- **RIFL/RamCloud**: add replicated shard groups with Raft consensus, linearizable RPC based on session leases. (J. Ousterhout/Stanford, 2014)
- **Spanner**. Scales up Thor's use of synchronized clocks for external consistency, timestamps for consistent snapshot reads in the past (2012).

Client/server transactions: another view

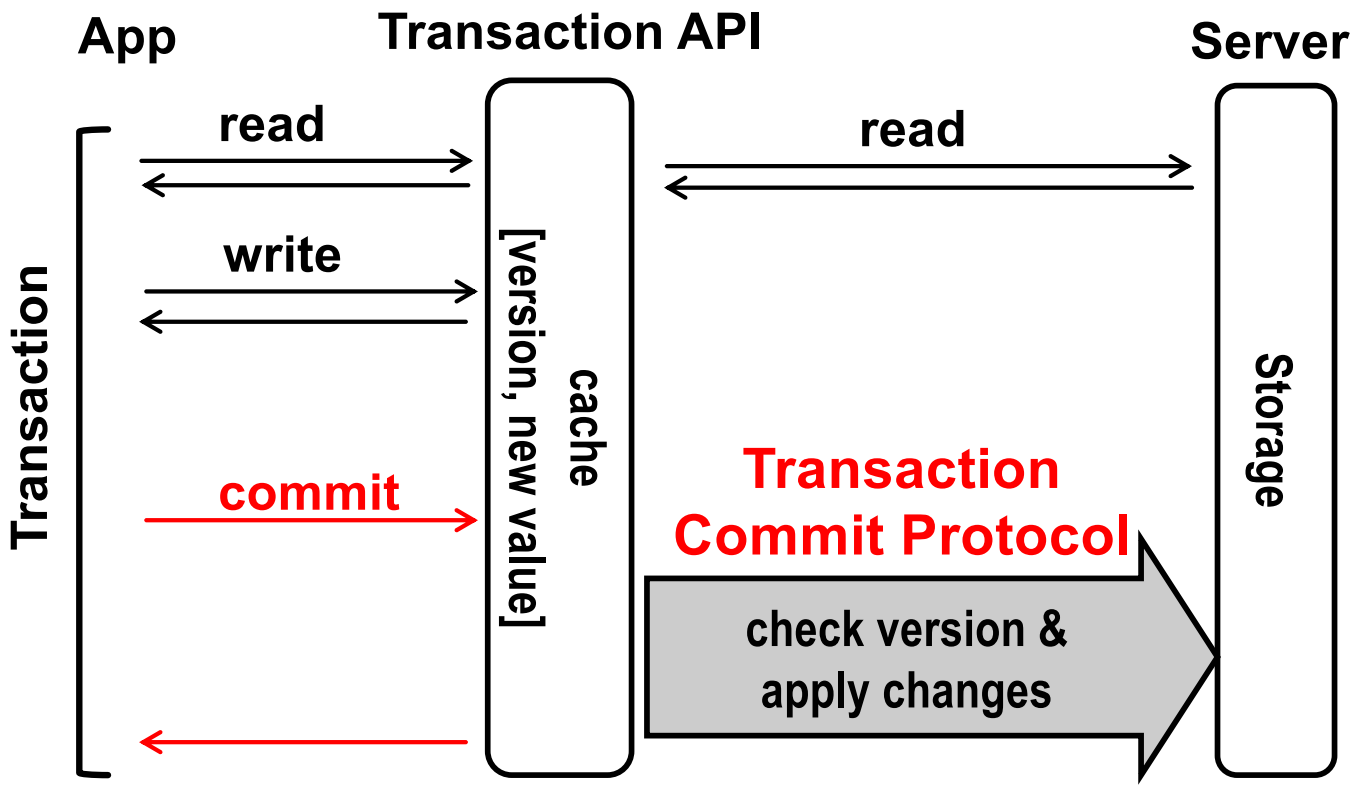
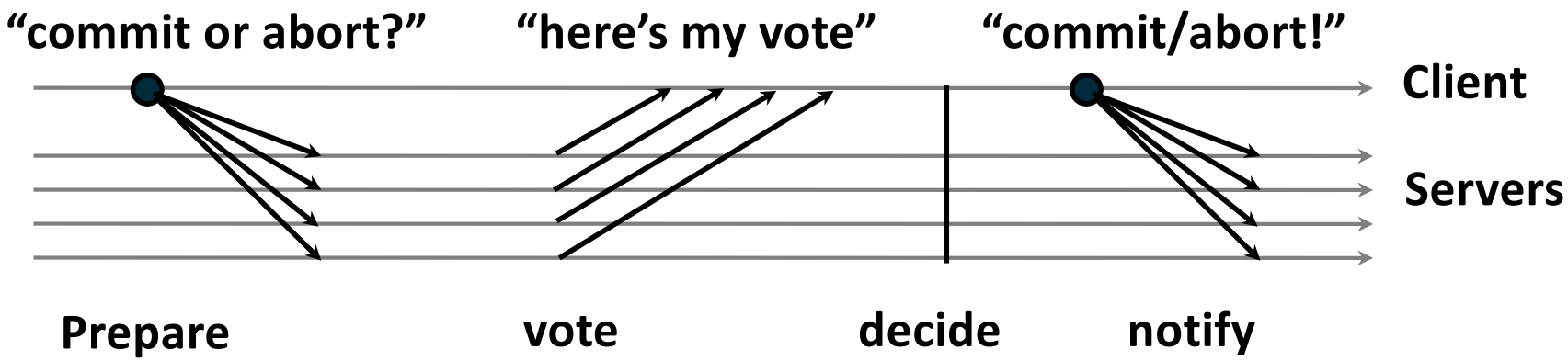


Image borrowed from authors of RamCloud/RIFL (SOSP 2015)

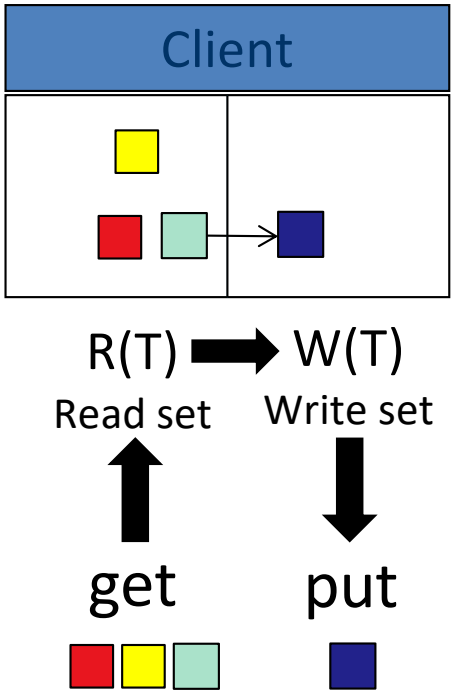
Two-phase commit (2PC)



- On **prepare**, servers **validate** for conflicts on shards.
- Conflicts? Abort mission.
- No conflicts? **Lock and log**, vote to commit.
- Wait for **decide**, release locks, log decision.

Spiffle: Prepare overview (at a shard)

- Objects have version timestamps.
  - Server returns timestamp for n on *get(n)*
  - Client passes timestamp back on *prepare*(*n*)
- **Validate**. Check timestamp for each object *n*.
  - Abort if current version is not expected version.
  - That means another transaction modified *n*.
- **Lock and Log**.
  - 2PL on all objects in R(T) and W(T)
  - Log locks: R(T) and W(T) pending decision
  - Log new values for objects in W(T)
  - Special case for **read-only** T: no locks



Spiffle: Prepare(n) at server

For an object *n* in R(T):

- **Step 1. Validate** *n* for T:
  - What version (or timestamp) of *n* did T read?
  - **Check version**. Does it match *n*'s current version? No → **Reject T**.
  - **Check lock**. Is *n* locked? Yes →**Reject T**.
- **Step 2. Lock and log** *n* for T:
  - Skip this step if T is rejected or T is read-only.
  - Lock *n* for T and log *n* as **prepared** for T.
  - If *n* is in W(T), also log new value for *n*.

Based on RamCloud/RIFL (SOSP 2015)

Spiffle: Decision

Log decision: commit or **abort**

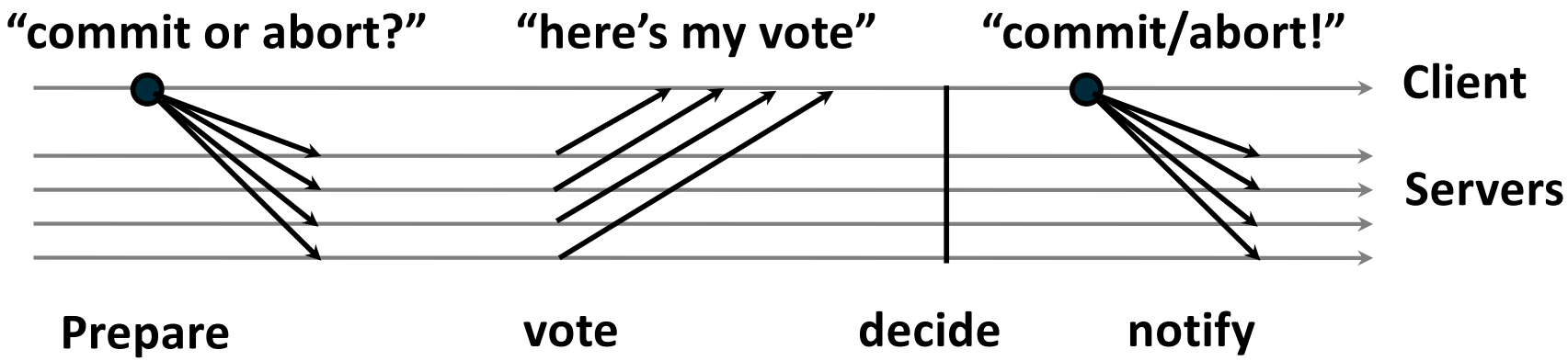
For each object *n* in R(T):

- If *n* is in W(T) and **commit**: update *n* and its version number
- Release lock on *n*

Note: no other transaction may see or touch *n* until T's outcome is known!

Based on RamCloud/RIFL (SOSP 2015)

Two-phase commit (2PC)



- 2PC requires a **unanimous** vote to commit.
- Shards prepare and vote independently and concurrently.
- What if shards don't agree on transaction order for conflicting S,T?
- S,T might prepare in different orders at different shards. **What then?**

Concurrent S and T with overlapping prepares

Suppose S and T are concurrent and conflicting. Then at least one of S and T aborts.

Case 1: neither S or T is read-only → both lock.

- Suppose S and T overlap at two objects m and n.
- Then **at least one of S and T aborts** (sees the other's locks).
  - If S prepares before T on either m or n, then T aborts.
  - If T prepares before S on either m or n, then S aborts.

→ If S and T prepare in different orders on m and n, then **both abort**.

Based on RamCloud/RIFL (SOSP 2015)

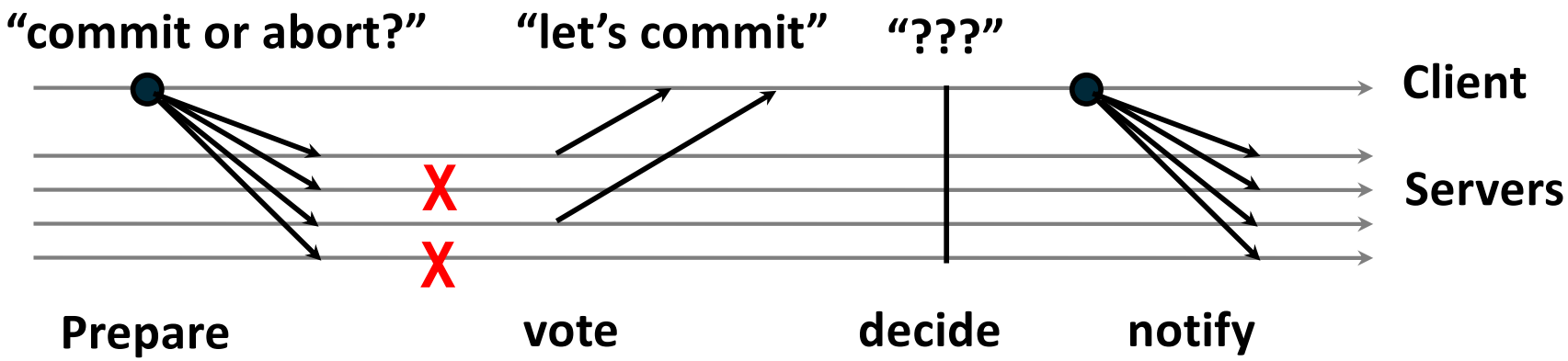
Concurrent S and T with overlapping prepares

Case 2: at least one of them (S) is **read-only**: no locks.

- Is T also read-only? → **No problem**.
- Suppose instead that T locks.
- Suppose S and T overlap at objects *m* and *n*.
- If S prepares before T on both *m* and *n* → no problem.
- If T prepares before S on either *m* or *n*, then S aborts (sees locks).

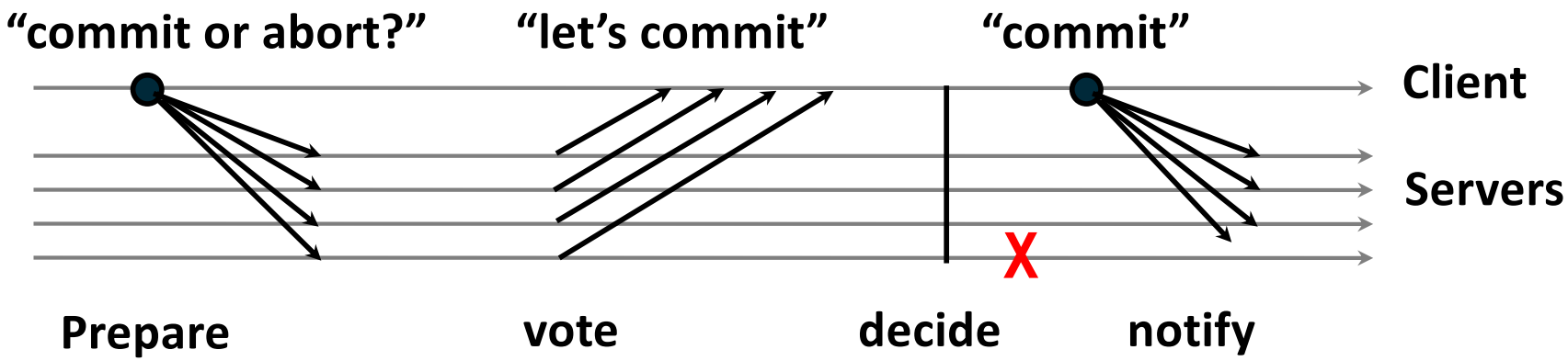
Based on RamCloud/RIFL (SOSP 2015)

2PC: what about failures?



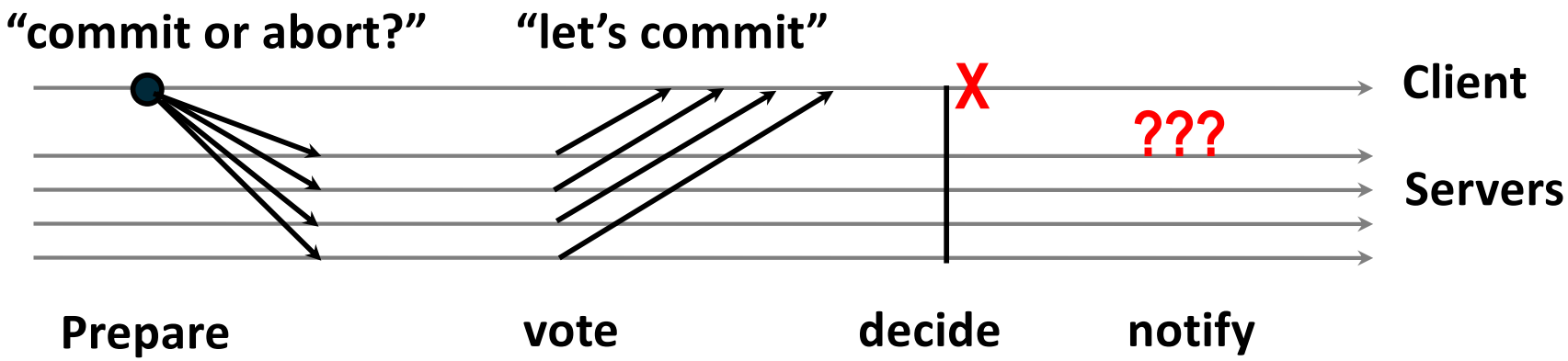
- 2PC requires a **unanimous** vote to commit.
- What should client (coordinator) do if a shard doesn't respond?

2PC: what about failures?



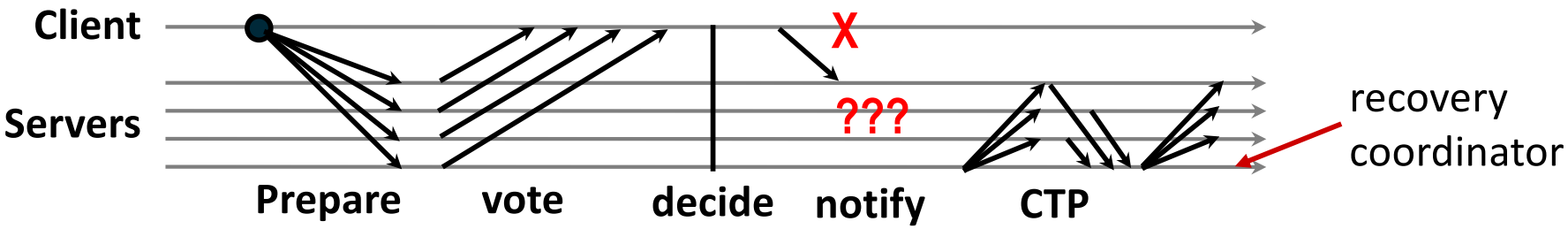
- What if a shard votes to commit, and then crashes?

2PC: what if client/coordinator fails?



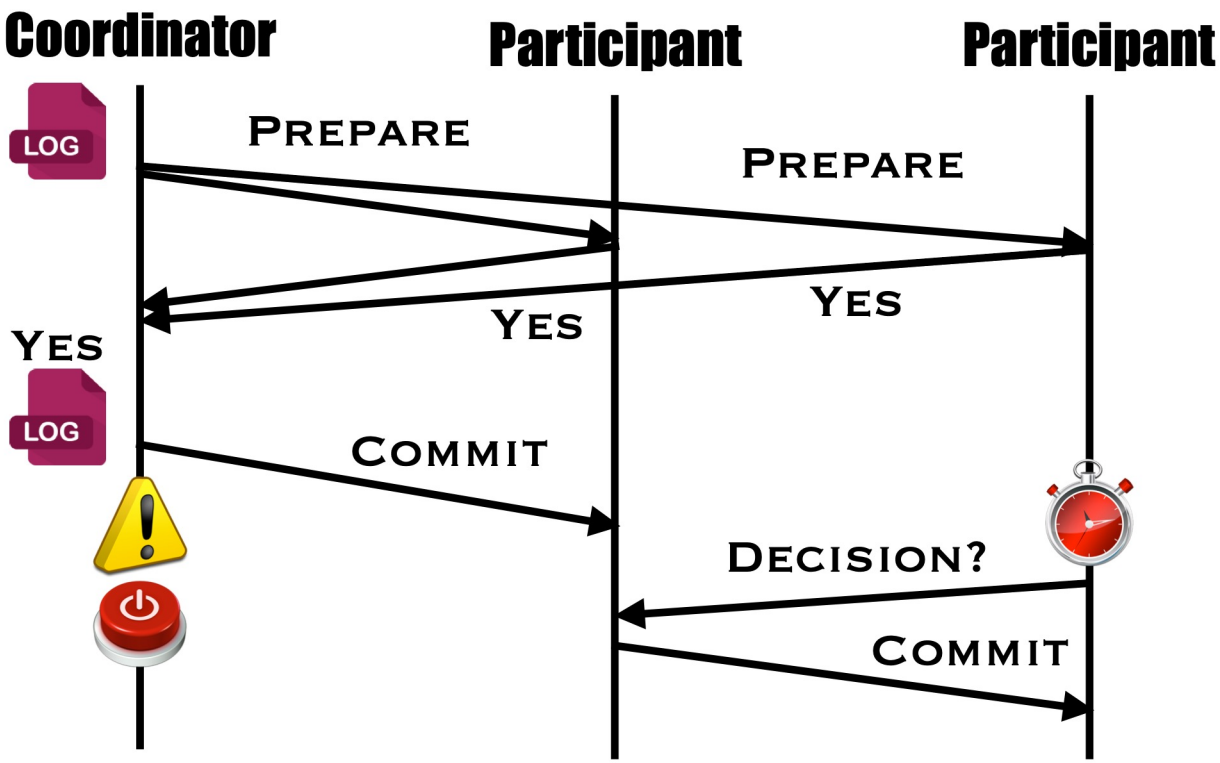
- It is possible that the client never recovers.
- If/when it does recover, what does it remember? **Nothing**.
  - (For clients in Spiffle.)

2PC: what if client/coordinator fails?



- **Stuck** until outcome is determined.
- **Cooperative Termination Protocol (CTP)**: servers talk to one another.
- One server may initiate protocol and act as **recovery coordinator**.
- Servers must agree on outcome—can't just abort unless sure that none have committed.

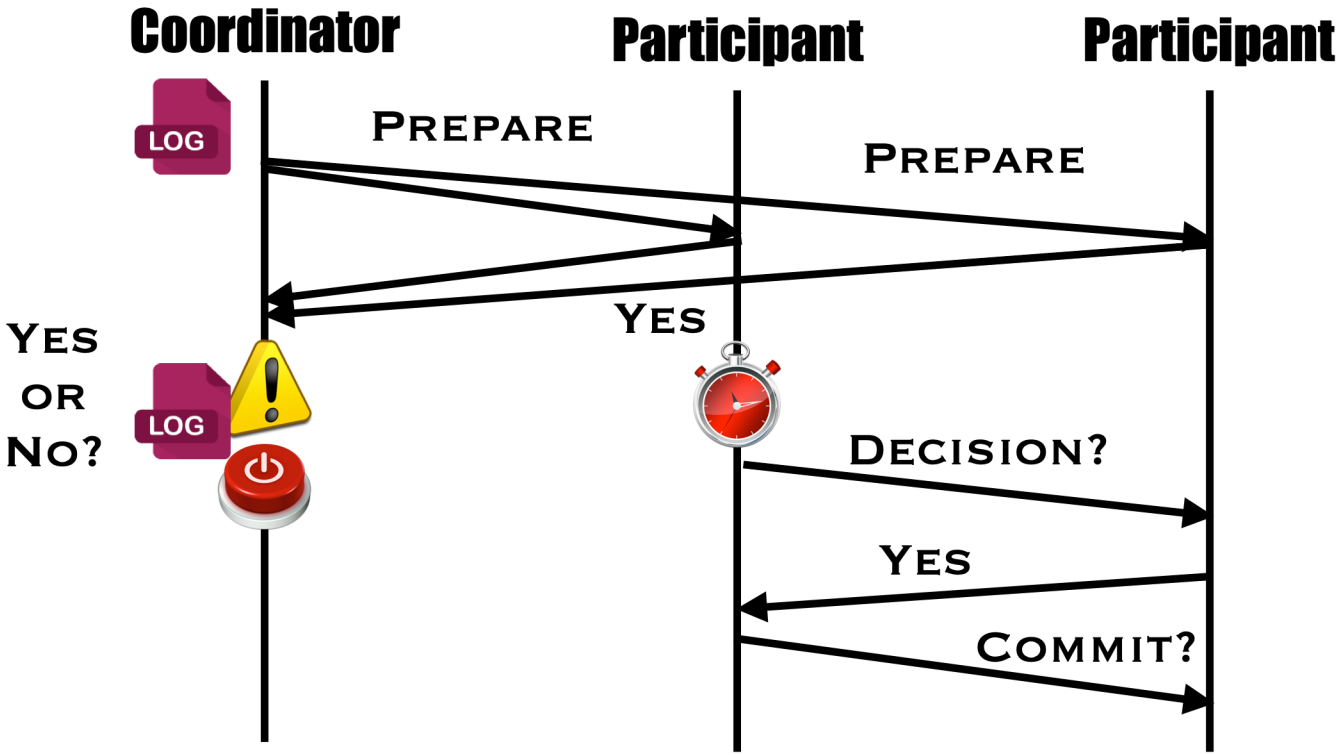
Do we need the coordinator?



In classical 2PC the coordinator logs and restarts 2PC protocol after recovering. We don't require that for cloud transactions: clients are "stateless".

https://courses.cs.washington.edu/courses/cse452/20wi/slides/l11-2PC.pdf

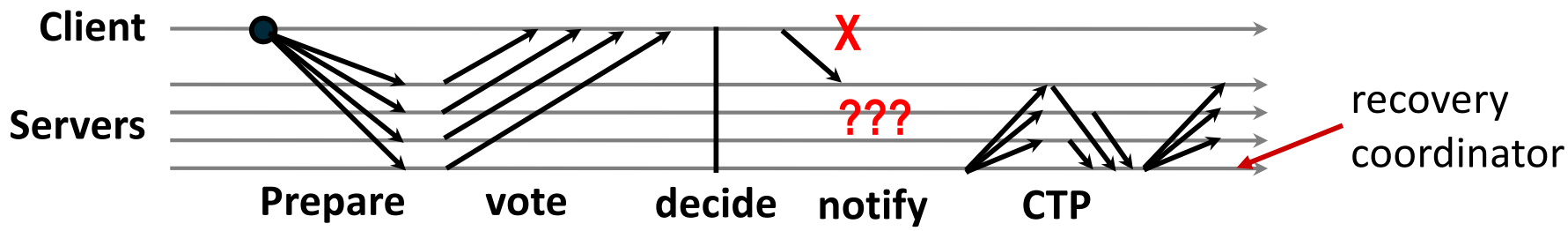
Can the Participants Decide Amongst Themselves?



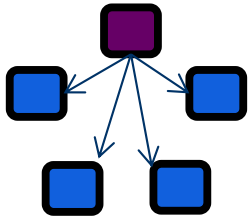
Servers/participants may decide outcome cooperatively if client/coordinator is failed/unreachable.

https://courses.cs.washington.edu/courses/cse452/20wi/slides/l11-2PC.pdf

Cooperative recovery: broad strokes



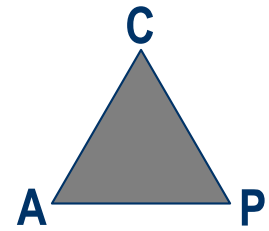
- **Beware** races between recovery coordinator(s) and/or client—if it restarts.
- Shards log relevant state (e.g., previous RPC replies) to handle race cases.
  - Details of **cooperative termination protocol** (CTP): see RIFL paper (out of scope)
- No access to prepared objects pending decision → unavailable.
  - **Safe but not live!**
- Solution: **replicated shard groups**
  - 2PC and replication: better together!



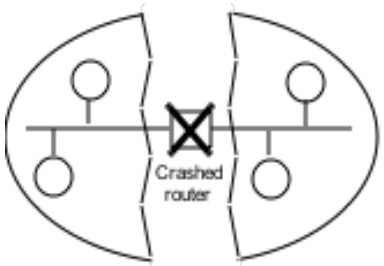
2PC is a consensus problem

Consensus can be safe or live but not both.

- In an asynchronous network with failures and **partitions**
- Safe (consistent) consensus may occur recognizably, but liveness (availability) is not guaranteed.
- Fischer-Lynch-Patterson (1985)
- Proof intuition: a "failed" server might just be "slow"—and may rise from the dead at an inconvenient time.



Choose two.



Network partition

FLP popularized as Brewer's Conjecture or CAP theorem. **No you can't have it all**: **C**onsistency + **A**vailability in presence of **P**artitions.